

Key Synchronization Architecture Specification for Quantum Key Distribution Networks

Abstract

This document proposes a distributed architecture for synchronizing key states between remote Key Management Entities (KMEs) in a Quantum Key Distribution (QKD) networks using the **Raft** consensus algorithm for key state consistency across the distributed network and **Zenoh** protocol at the network level for node communications.

1 Overview

This document aims to give a possible of many solutions for the main problem of Quantum Key Distribution (QKD) networks: **key synchronization**. As the architectures become more complex (multi-node and/or multi-vendor scenarios), the need of creating a system on which consistency across all the architectures is part of the equation appears.

This is not a real problem where a central server maintains this consistency, but this type of network (centralized) may not be suitable for a scenario where multiple vendors collaborate, where every vendor should be "equal" to others in the network (lately in this specification it will be seen that this is not the case, since some nodes will be more important for the Raft algorithm to work than others).

This is where distributed networks may come in handy, but the main problem in this situation is that the QKD networks need consistency among all the nodes to work properly. Consistency in a distributed network may be a challenge, but several solutions have been developed to this problem. One of them are **consensus algorithms**.

This type of algorithms are key in distributed networks, ensuring that nodes in the network agree in a common state of the system. They are largely used in blockchain networks: Proof of work (PoW) or Proof of Stake (PoS) are some examples of these algorithms [1].

Raft algorithm is defined by his authors as "*a consensus algorithm for managing a replicated log*" [2]. This may not be suitable for a QKD network at first, but here is the catch: the Raft algorithm will be used for keeping track of the **state** of the keys among all the network; the actual key data will be exchanged only between the nodes that want to communicate via API REST defined in the **ETSI-020** draft.

A node that does not participate in the communication **will not have access to the key material: only to the state of a related key**. This is an important point, since if every node in the network would have access to all the key material, if this node gets compromised, all the network will be in danger.

Zenoh protocol is a pub/sub/query protocol. It is scalable and fast. In this architecture, Zenoh will be used to transport some relevant (meta)data (TBD), to make nodes auto-discoverable between them and so nodes that are not connected directly know a path to each other, among other possibilities (TBD)

2 Assumptions

This specification makes the following assumptions regarding the presented scenario:

- All nodes in the network are considered properly secured. Although a node may be compromised in a real scenario, this specification does not cover that situation.
- No configuration changes are done in the Raft cluster. This may present a problem, but this is already covered by the authors of the algorithm in the original tech report [2, p. 11].
- No log compaction will be needed. As the nodes interact, the logs in the Raft cluster will extend and cause delay issues. This will not be considered in this specification, but this is already covered by the authors of the algorithm in the original tech report [2, p. 11, 12].
- Leader election is out of the scope. The specification assumes that a valid leader is elected, available, and ready to answer client requests.

3 Raft consensus algorithm

As defined before, Raft is defined by his authors as “*a consensus algorithm for managing a replicated log*”. Its authors created it in order to have a less-complex consensus algorithm than Paxos, a foundational consensus algorithm presented by Leslie Lamport in the 90s [3]. This section aims to give a quick overview of the Raft algorithm to understand its implementation in the QKD network. All the information can be expanded in the tech report “**In Search of an Understandable Consensus Algorithm**” [2].

Raft’s main design goal was to create a more understandable consensus algorithm, since Paxos is highly abstract and difficult to understand, even from its original article “*The Part-Time Parliament*” [3].

The main Raft structure is the **Raft cluster**. A Raft cluster is a collection of 3-7 nodes (an odd number is preferred) where a node is elected as “leader” of the cluster. This makes Raft a **leader-based** algorithm. The rest of the nodes in the cluster are called **followers**, and the rest of the nodes in the network that does not belong to the Raft cluster are called **clients**.

Raft algorithm guarantees that each of the following properties are true at all times:

- **Election Safety**: At most one leader can be elected in a given term.
- **Leader Append-Only**: A leader never overwrites or deletes entries in its log: it only appends new entries.
- **Log Matching**: If two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index.

- **Leader Completeness:** If a log entry is committed in a given term, that entry will be present in the logs of the leaders for all higher numbered terms.
- **State Machine Safety:** If a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index.

Within the Raft cluster, a node may have the following states:

- **Follower:** Passive node (in the Raft context, not in the QKD) that responds to candidates and leader requests.
- **Candidate:** Node enrolled in an election process to be promoted to leader of the cluster.
- **Leader:** Node that handles all client requests. If client contacts a follower, the follower will redirect the client to the leader.

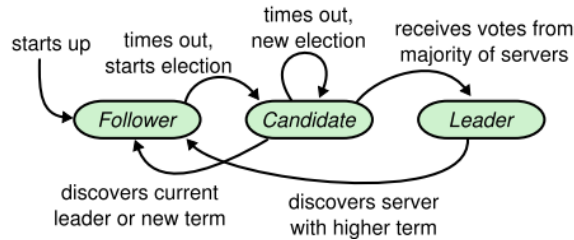


Figure 1: Node states in the Raft cluster. [2]

Leader election method is out of the scope of this specification. More information can be found about the leader-election process in the tech report.

Once a leader has been elected, it will start to receive client requests. Once a request is received, the leader appends the action in its log as a new entry, and starts the replication procedure in the followers among the cluster. Once a majority of the cluster has safely replicated the entry and the leader has been notified, this action becomes **committed**, and notifies the client of the outcome. If followers crash or run slowly during the replication procedure, the leader will retry the replication in those followers, even after the action is committed and the client has been notified.

Once an action is committed, this action is considered as **the truth** in the cluster. No follower, client or any node can contradict this action.

4 Zenoh protocol

The creators of Zenoh define it as a “pub/sub/query protocol unifying data in motion” [4].

It allows communications in this pub/sub/query model, similar to protocols like MQTT; but where MQTT uses a broker centralized architecture, where every message goes through the broker acting as a “router” for the messages between publishers and subscribers, Zenoh uses a decentralized architecture, where there are no dependencies for intermediate brokers.

Zenoh supports several configuration types:

- Peer to peer
- Client
- Peers mesh
- Zenoh router.

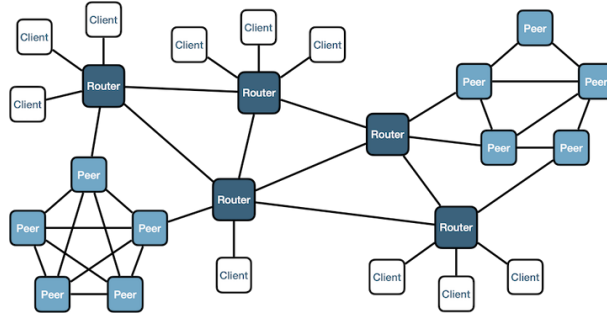


Figure 2: Every configuration type available in Zenoh. Peer to peer (bottom left), client, peers mesh (top right) and router. [4]

These configurations can be interconnected. In a multi-vendor scenario, every configuration can be deployed, although two are more suitable:

- Peers mesh inter-connect peers in the same local network, without the need of linking every peer to other peer.
- Zenoh router is suitable if there is no link between peers of different vendors.

Client configuration is similar to the broker architecture in MQTT, and Peer to peer may add overhead, as links have to stay up for every node connected.

5 Raft and Zenoh in a QKD network

5.1 Using Raft

In order to adapt a QKD node to the Raft algorithm, a **Raft module** will need to be added to it.

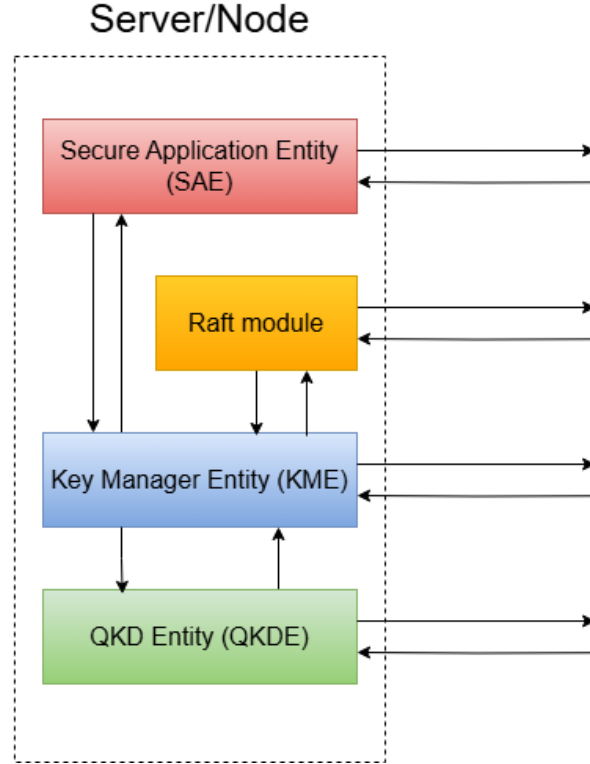


Figure 3: Node with Raft module

The Figure 3 illustrates how the QKD node structure and its communications would be: The Raft module would only communicate to the internal Key Manager Entity (KME) and the exterior of the node, with other Raft modules in the interconnected nodes.

As described in the Raft algorithm section (check 3), a node can be part of the cluster, taking three possible roles (follower, candidate and leader) or be just a client of the Raft cluster. Every Raft module may implement all features, in case it needs them, but **not all QKD nodes would be part of the Raft cluster**. Only selected modules will be part of the Raft cluster. In order to distinguish this nodes in the figures, this schema will be followed (candidates nodes will not be defined, since the specification assumes that the leader has been elected already):

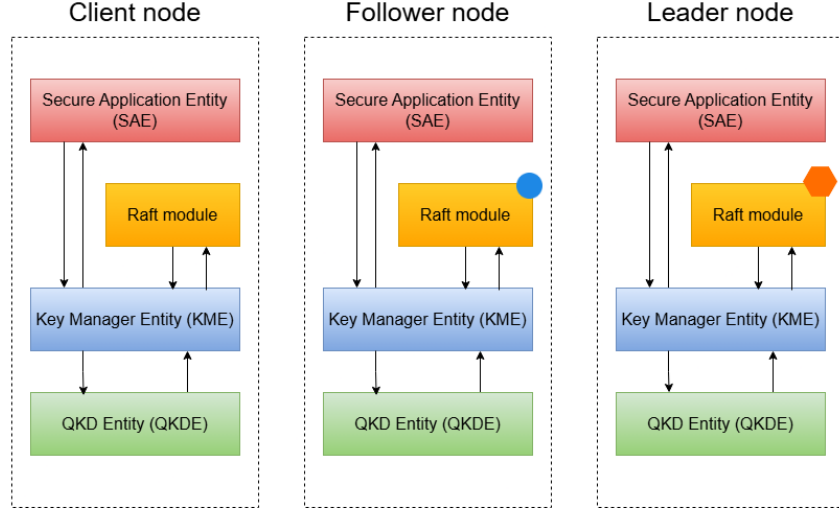


Figure 4: Types of QKD+Raft nodes

Also, in order to work with the Raft algorithm, a state machine must be defined. The state machine in this case would be a simple linear state machine (in other words, when a state has been reached, there is no going backwards) with the possible states of the keys:

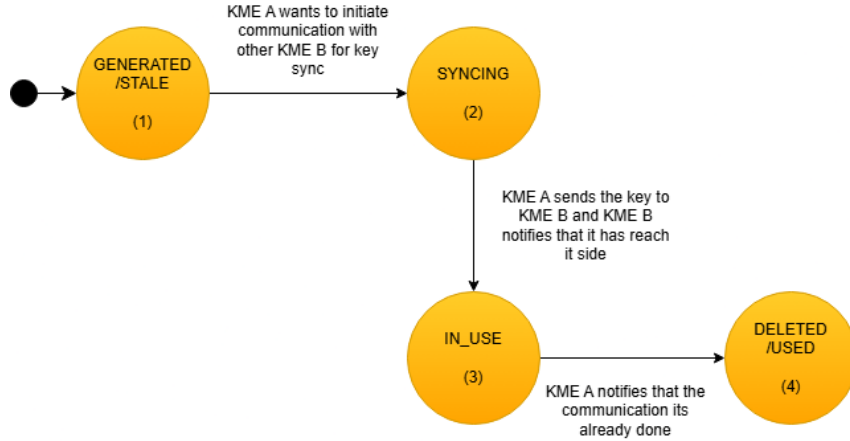


Figure 5: Linear state machine containing name and ID number of state

The work of the Raft module will vary, depending on the type of the QKD node:

- **Client** nodes' Raft module will be in charge of communicating local KME's **in-tention** (without performing it) to the leader of the Raft cluster. This intention comprehends any action that forces a change in the state of a key, described in Figure 5.
- **Follower and leader** nodes' Raft module will behave like they would do in the Raft cluster: replicate logs or attend clients, respectively.

Message contents are still TBD, but as stated in the overview, no key material would be in the messages. An example of a message that KME A would send if it wants to start a communication with KME B could be the following:

```
1 SYNC req_id="938f8017-364d-4ebd-9af6-9b33a81af128"
2 key_id="43dbfdf3-269c-4bec-9f6c-6695e5e3c1e3" master="A" slave="B"
```

Until the leader of the Raft cluster has not answered this SYNC with the outcome of this request, KME A **shall not initiate** any attempt of syncing the key with KME B.

When the leader receives this request, and following the example, it would check if this transaction is possible. This means answering the following questions:

- Has been the key with key ID “43dbfdf3-269c-4bec-9f6c-6695e5e3c1e3” generated within “A”?
- Has been the key with key ID “43dbfdf3-269c-4bec-9f6c-6695e5e3c1e3” and master “A” been used in other communication before?
- Has slave KME , “B”, any key with the same ID?

Note: It is true that the ID of a key may be the same for the same master, but if, like in the example, UUIDv4 is used, it is more probably that the KME is re-using some stored key than two UUIDv4 generated are the same within the same machine.

If the answer to the first question is “Yes” and the answer to any of the other questions is “No”, then the communication can proceed. The leader then will store this action in its logs and it would begin the replication process to its followers. Once most followers in the cluster has acknowledge the action, the leader will response to the client with an acceptance message (fully contents TBD). An example of what this message’s content could be:

```
1 SYNC req_id="938f8017-364d-4ebd-9af6-9b33a81af128"  
2 status="accepted"
```

In this transaction, following the state machine in Figure 5, the key would have passed from the “GENERATED/STALE” state to the ”SYNCING” state. Once the client has received the acceptance message, the transaction can be performed.

In case that:

- Key generation has not been notified to Raft cluster before trying to use it, so it does not have reached the first ”GENERATED/STALE” state.
- Master KME is trying to re-use a old key.
- Slave KME cannot store that key due to matching IDs from another KME
- Generated “req_id” has been already used

the leader would response the client with a non-acceptance message (fully contents TBD). For example, if the slave KME cannot store that key ID, an example of this message could be:

```
1 SYNC req_id="938f8017-364d-4ebd-9af6-9b33a81af128"  
2 status="non-accepted" reason="SLAVE_MATCHING_KEYID_ERROR"
```

Then, it would be KME’s responsibility to initiate another transaction without errors. In the case of the example, it will generate another key ID for that key, and retry it.

This procedure can be extended to all transactions of the state machine, although only the first contact would need explicit permission from the leader to proceed, since the ”SYNCING”, ”IN_USE” and ”DELETED/USED” states are formally informational states, so other nodes can keep track of its state.

The main idea of using Raft in the architecture is to introduce an orchestrator in the distributed network, without modifying the already existent standards for the QKD nodes interactions.

5.2 Using Zenoh

In the QKD architecture, Zenoh will be in charge of two tasks:

- Auto-discovering of new KMEs
- Routing secured messages through the Zenoh network (both Raft and ETSI 020 messages)

As for the first task, Zenoh provides two different auto-discovering methods:

- **Scouting:** Multicast communications on multicast group 224.0.0.224 port UDP/7446 to discover neighbors through scout messages. Used when a node has just connected to the network.
- **Gossip:** When a new node has been scouted or there is a change in the network topology, all the information about routing that the node possess is forwarded to its neighbor node. Then, the neighbor node updates its routing table.

As from the second task, it can be done thanks to the internal routing table that Zenoh provides. If connected to at least one node in the network, the message will reach its destination. Zenoh also supports hop-by-hop TLS, so the messages will be secured.

Zenoh offers two different ways of communicating the data:

- **Pub/sub:** One of the nodes publishes into a topic, and subscribers for this topic ingest the data.
- **Storage/Query:** One of the nodes store the info in a topic. When another node wants this data, it queries the node for the topic.

By the nature of the ETSI-020 communications, where the master is the one who initiates the communication, a mix of the two communications are used. The mapping REST endpoint → Zenoh communication:topic is the following:

- /kmapl/version → Storage/Query:{master_node_hostname}/kmapl/version
- /kmapl/v1/ext_keys → Pub/Sub:{slave_node_hostname}/kmapl/ext_version
- /kmapl/v1/ext_keys/void → Pub/Sub:{slave_node_hostname}/kmapl/ext_version/void
- /kmapl/v1/ext_keys/ack → Pub/Sub:{master_node_hostname}/kmapl/ext_version/ack

The data will maintain the JSON format, since Zenoh supports it.

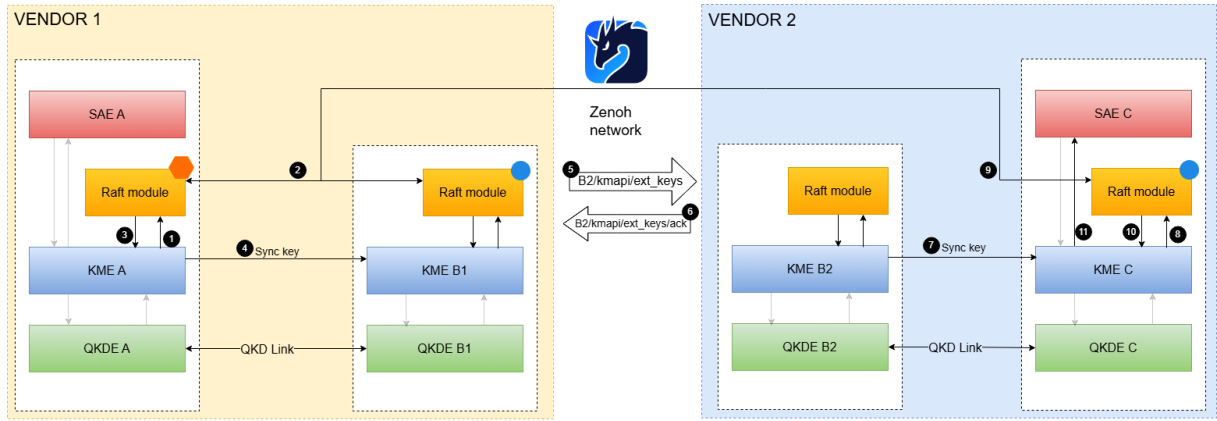


Figure 6: Communication example

6 Communication example

In this section, a fully communication example is performed. The same network schema from the Figure 2 of the ETSI 020 draft is used. Once **SAE A** has notified its intention on communicate with **SAE C**

- **Step 1:** **KME A** sends the request to its Raft module to start a communication with **KME C**.
- **Step 2:** Before sending the OK to its KME, the leader sends the event to its follower. The key now is in the **SYNCING** state.
- **Step 3:** The Raft module from **KME A** communicate to the KME that the process can start.
- **Step 4:** **KME B1** obtain the key through the QKD modules that were established over the QKD link.
- **Step 5-6:** **KME B1** publishes the key in the **KME B2** in its topic, and **KME B2** ingest that data. **KME B2** publish into **KME B1** ack topic to acknowledge that the key has been received.
- **Step 7:** **KME C** obtain the key through the QKD modules that were established over the QKD link.
- **Step 8-9:** Before sending the key to its SAE, **KME C** communicates to its Raft module that it has received the keys. Since the raft module is a follower in this schema, it communicates to the leader the new state. The leader appends it and allows it since it has been acknowledge by the rest of the followers. The key is now in the **IN_USE** state.
- **Step 10-11:** The raft module communicates **KME C** that the new state has been acknowledge and that it can proceed. **KME C** communicates its SAE the key.

Once the communication between the SAEs its done, the keys are voided following the same communication schema. Once finished, the key is in the **DELETED/USED** state.

References

- [1] Md Sadek Ferdous, Mohammad Javed Morshed Chowdhury, and Mohammad A. Hoque. “A survey of consensus algorithms in public blockchain systems for cryptocurrencies”. In: *Journal of Network and Computer Applications* 182 (2021), p. 103035. ISSN: 1084-8045. DOI: [10 . 1016 / j . j n c a . 2021 . 103035](https://doi.org/10.1016/j.jnca.2021.103035). URL: [https : / / www . sciencedirect . com / science / article / pii / S1084804521000618](https://www.sciencedirect.com/science/article/pii/S1084804521000618).
- [2] Diego Ongaro and John Ousterhout. *In Search of an Understandable Consensus Algorithm (Extended Version)*. Tech. rep. Stanford University, 2014.
- [3] Leslie Lamport. “The part-time parliament”. In: *ACM Trans. Comput. Syst.* 16.2 (May 1998), pp. 133–169. ISSN: 0734-2071. DOI: [10 . 1145 / 279227 . 279229](https://doi.org/10.1145/279227.279229). URL: <https://doi.org/10.1145/279227.279229>.
- [4] ADLINK. *Zenoh - The Zero Overhead, Pub/Sub, Store, Query and Compute Protocol*. 2022. URL: <https://zenoh.io/>.